

An Exploration of K-Means Clustering Optimizations

Molly Pate

Spring 2026

Motivation

- K-means is widely used in unsupervised learning
- Computationally expensive for large datasets
- CIFAR-10: 60,000 images, 3072-D feature vectors
- Distance computation dominates runtime ($O(N \times K \times \text{DIM})$)
 - N = total data points
 - K = 10 (number of clusters)
 - DIM = point dimensionality (3072 for CIFAR-10 points)
- Goal: accelerate using GPU parallelism

airplane



automobile



bird



cat



deer



dog



frog



horse



ship



truck



Problem Overview



Standard K-means workflow:

- Initialize K centroids
- Assign points => nearest centroid
- Recompute centroids (mean)
- Repeat until convergence



Main cost: repeated distance calculations



High dimensionality increases compute + memory load

CPU Baseline

- Fully sequential implementation
- Nested loops over:
 - data points
 - centroids
 - dimensions
- Full dataset used every iteration
- No parallelism
- Serves as performance baseline

GPU Standard (Batch) Implementation

- One thread per data point (assignment step)
- Parallel distance computation across centroids
- Shared memory tiling for centroid reuse
- Atomic operations for centroid updates
- Goal: maximize parallel throughput

Initial Bottlenecks

- Atomic contention during centroid updates
- High global memory bandwidth usage
 - Couldn't allocate on the host in shared memory
- 3072-D vectors increase memory cost
- Synchronization overhead (`__syncthreads()`)
- Kernel becomes partially memory-bound

Mini-Batch Implementation

- Uses random subset of data each iteration
- Reduces full-dataset computation cost
- Incremental centroid updates (learning rate)
- Faster per-iteration execution
- Tradeoff: slight approximation error

Mini-Batch Bottlenecks

- Reduced computation per iteration
- Still uses atomic updates for centroids
- Frequent updates increase synchronization cost
- Contention remains ($K = 10$ shared clusters)
- Bottleneck shifts, not eliminated

Key Design Decisions

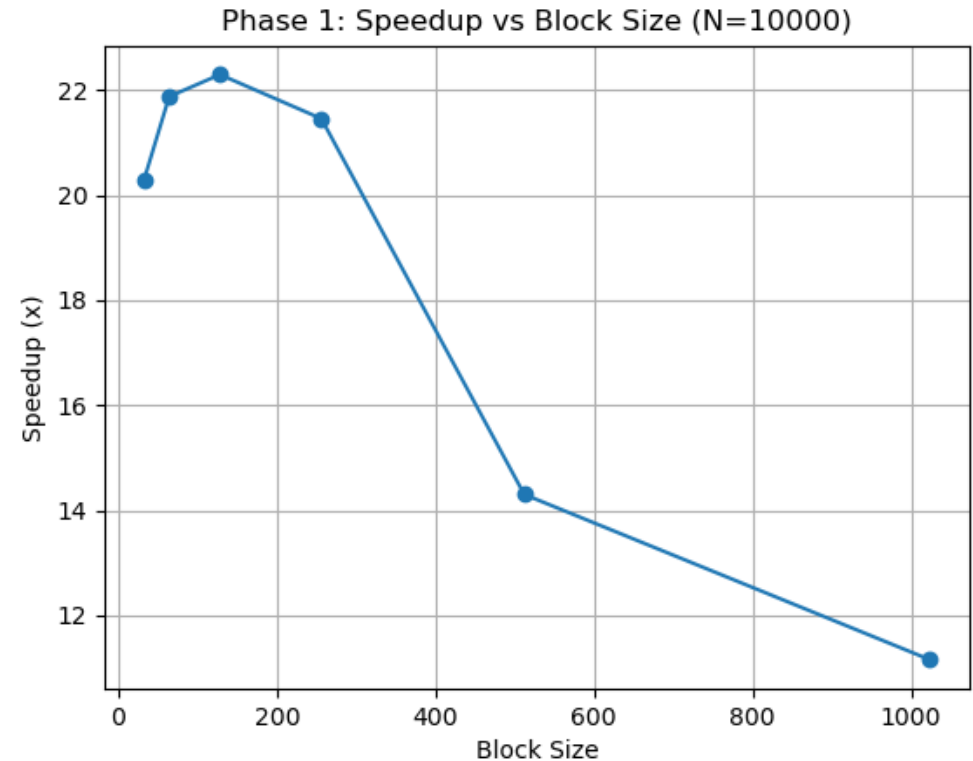
- One thread per data point
 - simple + scalable
- Shared memory tiling
 - reduce global memory access
- Atomic updates
 - avoid complex reductions
- Mini-batching
 - reduce dataset-wide passes
- Fixed $K = 10$
 - optimized for CIFAR-10 structure

Benchmark Setup

- Benchmark designed in 4 controlled phases:
 - **Block size scaling** (N = 10,000)
 - block sizes = 32, 64, 128, 256, 512, 1024
 - **Standard Implementation scaling:** (block size = 256)
 - N = 1000, 10000, 50000, 100000, 1000000
 - **Mini-Batch Scaling** (block size = 256)
 - N = 1000, 10000, 50000, 100000, 1000000
 - **Stress test** (N = 1M)
 - block sizes = 128, 256, 512
- Outputs:
 - Runtime measurements (CPU vs GPU)
 - Cluster assignment CSV files for analysis

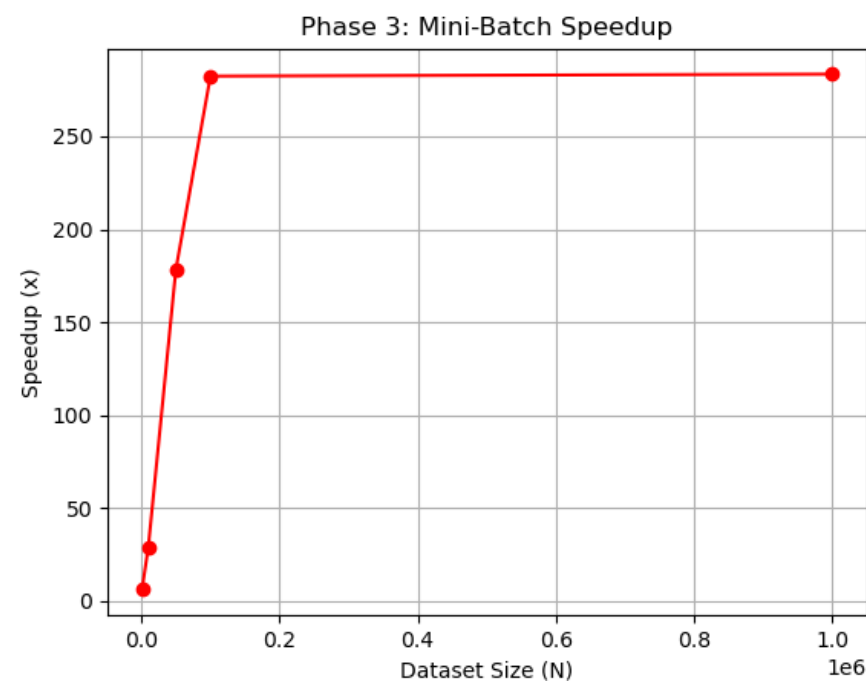
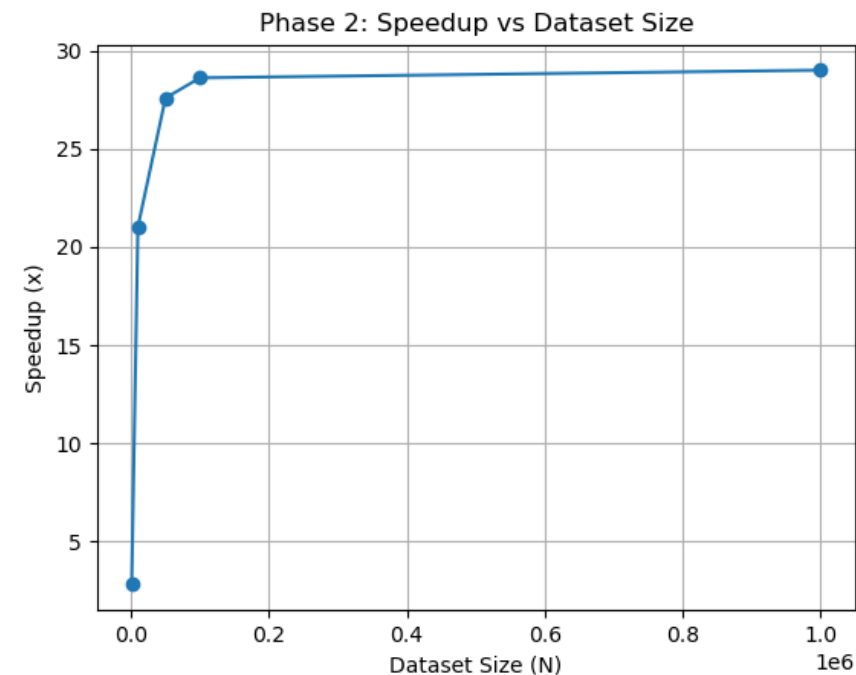
Block Size Scaling

- Best speedup at block size 128 (~22x)
- Large blocks reduce performance due to sync + atomic overhead
- Takeaway:
 - Optimal block size balances occupancy and synchronization cost



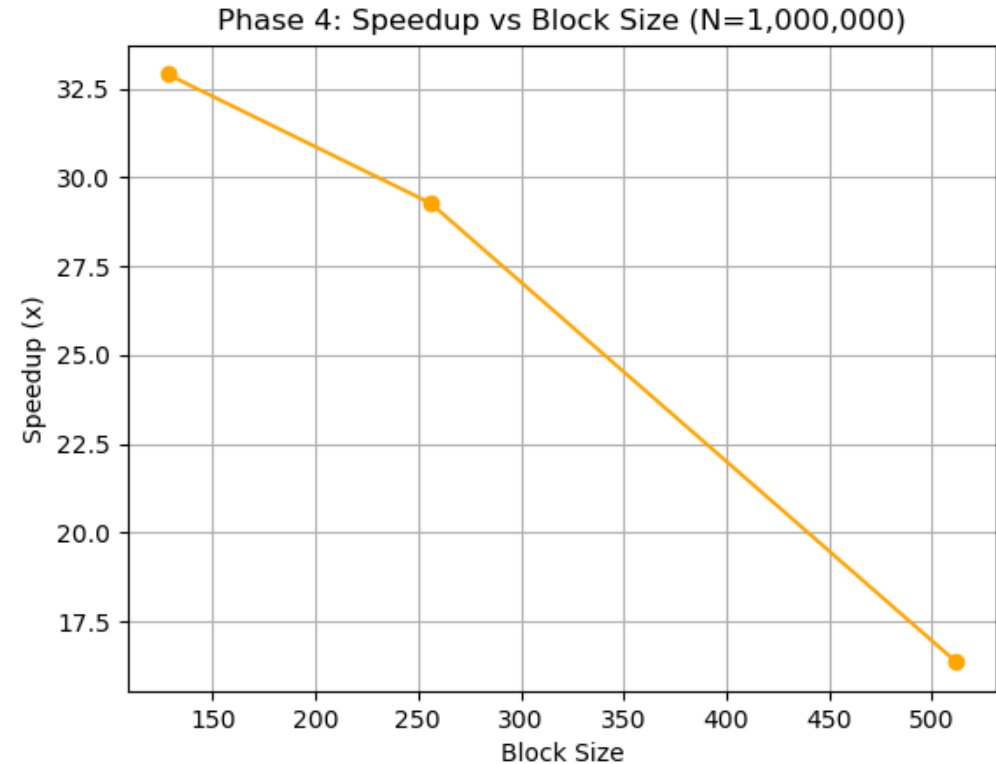
Standard Batch vs Mini-Batch Scaling

- Speedup grows to ~29x, then plateaus
- Gains slow at large N due to memory + atomic contention
- Takeaway:
 - GPU becomes memory/synchronization bound, not compute bound
- Mini-batch reaches up to 283x speedup
- Much lower per-iteration cost and memory usage
- Takeaway:
 - Major gains come from reducing workload, not just parallelism



Block Size Scaling on Large Data Sizes

- Best performance at block size 128 (~32.9x)
- Larger blocks hurt performance due to resource + contention limits
- Takeaway:
 - At scale, performance is limited by contention, not compute capacity



Future Improvements

- Replace atomics with hierarchical reductions
- Improve mini-batch sampling (shuffle-based)
- Model more of the impact of mini-batch sizes on correctness performance
- GPU k-means++ initialization (or other heuristic)
 - To initialize K clusters

Conclusion

- GPU K-means achieves major speedup over CPU (up to ~30x standard, ~280x mini-batch)
- Performance is strongly affected by:
 - memory bandwidth limits
 - atomic synchronization (centroid updates)
 - block size and occupancy tradeoffs
- Standard implementation scales well but plateaus due to contention and memory pressure
- Mini-batch dramatically improves throughput by reducing per-iteration workload
- Takeaway:
 - Best performance comes from combining parallelism + memory optimization + algorithmic approximation, not parallelism alone

Thank you

References

- <https://codesignal.com/learn/courses/unsupervised-learning-and-clustering/lessons/enhancing-machine-learning-expertise-mini-batch-k-means-clustering-explained>
- <https://www.geeksforgeeks.org/machine-learning/k-means-clustering-introduction/>